C++ Pointers

Pointers

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.

The symbol of an address is represented by a pointer. In addition to creating and modifying dynamic data structures, they allow programs to emulate call-by-reference. One of the principal applications of pointers is iterating through the components of arrays or other data structures. The pointer variable that refers to the same data type as the variable you're dealing with has the address of that variable set to it (such as an int or string).

- **Syntax**

datatype *var_name;

int *ptr; // ptr can point to an address which holds int data

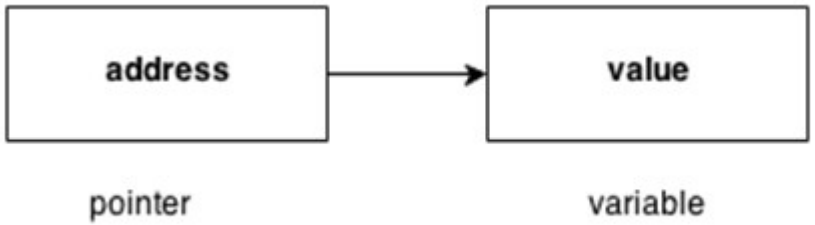
How to use a pointer?

Establish a pointer variable.

employing the unary operator (&), which yields the address of the variable, to assign a pointer to a variable's address.

Using the unary operator (*), which gives the variable's value at the address provided by its argument, one can access the value stored in an address.

Since the data type knows how many bytes the information is held in, we associate it with a reference. The size of the data type to which a pointer points is added when we increment a pointer.



Advantage of pointer

1) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.

2) We can return multiple values from function using pointer.

3) It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usage of pointers in C++ language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
∗ (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C++ language can be declared using ∗ (asterisk symbol).

int ∗ a; //pointer to int

char ∗ c; //pointer to char

Pointer Example

- Let's see the simple example of using pointers printing the address and value.

```
#include <iostream>
using namespace std;
int main()
{
int number=30;
int *   p;
p=&number;//stores the address of number variable
cout<<"Address of number variable is:"<<&number<<endl;
cout<<"Address of p variable is:"<<p<<endl;
cout<<"Value of p variable is:"<<*p<<endl;
  return 0;
}
```

Output:

Address of number variable is:0x7ffccc8724c4
Address of p variable is:0x7ffccc8724c4
Value of p variable is:30

Pointer Program to swap 2 numbers without using 3rd variable

```
#include <iostream>
using namespace std;
int main()
{
int a=20,b=10,*p1=&a,*p2=&b;
cout<<"Before swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
*p1=*p1+*p2;
*p2=*p1-*p2;
*p1=*p1-*p2;
cout<<"After swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
  return 0;
}
```

Output:

Before swap: *p1=20 *p2=10

After swap: *p1=10 *p2=20

What is a void pointer?

This unique type of pointer, which is available in C++, stands in for the lack of a kind. Pointers that point to a value that has no type are known as void pointers (and thus also an undetermined length and undetermined dereferencing properties).

This indicates that void pointers are very flexible because they can point to any data type. This flexibility has benefits. Direct dereference is not possible with these pointers.

Before they may be dereferenced, they must be converted into another pointer type that points to a specific data type.

What is a invalid pointer?

A pointer must point to a valid address, not necessarily to useful items (like for arrays). We refer to these as incorrect pointers. Additionally, incorrect pointers are uninitialized pointers.

```
int *ptr1;
int arr[10];
int *ptr2 = arr+20;
```

Here, ptr1 is not initialized, making it invalid, and ptr2 is outside of the bounds of arr, making it likewise weak. (Take note that not all build failures are caused by faulty references.)

What is a null pointer?

- A null pointer is not merely an incorrect address; it also points nowhere. Here are two ways to mark a pointer as NULL:
- **int** *ptr1 = 0;
- **int** *ptr2 = NULL;

What is a pointer to a pointer?

In C++, we have the ability to build a pointer to another pointer, which might then point to data or another pointer. The unary operator (*) is all that is needed in the syntax for declaring the pointer for each level of indirection.

char a;

char *b;

char ** c;

a = 'g';

b = &a;

c = &b;

Here b points to a char that stores 'g', and c points to the pointer b.

What are references and pointers?

- Call-By-Value
- Call-By-Reference with a Pointer Argument
- Call-By-Reference with a Reference Argument

Call by Value

```
#include <iostream>
using namespace std;

void changeValue(int x) {
    x = 100;
}

int main() {
    int a = 10;
    changeValue(a);
    cout << a;
    return 0;
}
```

- Output: 10

Call by reference value

```
#include <iostream>
using namespace std;

void changeValue(int &x) {
    x = 100;
}

int main() {
    int a = 10;
    changeValue(a);
    cout << a;
    return 0;
}
```

- Output: 100

```
#include <iostream>
using namespace std;

void changeValue(int *x) {
    *x = 100;  // Modifies the original variable through the pointer
}

int main() {
    int a = 10;
    changeValue(&a);  // Pass the address of 'a'
    cout << "After function call: " << a << endl;  // Output: 100
    return 0;
}
```

```
#include
using namespace std;
// Pass-by-Value
int square1(int n)
{cout << "address of n1 in square1(): " << &n << "\n";
n *= n;
return n;
}
// Pass-by-Reference with Pointer Arguments
void square2(int* n)
{
cout << "address of n2 in square2(): " << n << "\n";
*n *= *n;
}
```

```
// Pass-by-Reference with Reference Arguments
void square3(int& n)
{

cout << "address of n3 in square3(): " << &n << "\n";
n *= n;
}
void example()
{
   // Call-by-Value
   int n1 = 8;
   cout << "address of n1 in main(): " << &n1 << "\n";
   cout << "Square of n1: " << square1(n1) << "\n";
   cout << "No change in n1: " << n1 << "\n";
```

```
 // Call-by-Reference with Pointer Arguments
    int n2 = 8;
    cout << "address of n2 in main(): " << &n2 << "\n";
    square2(&n2);
    cout << "Square of n2: " << n2 << "\n";
    cout << "Change reflected in n2: " << n2 << "\n";

    // Call-by-Reference with Reference Arguments
    int n3 = 8;
    cout << "address of n3 in main(): " << &n3 << "\n";
    square3(n3);
    cout << "Square of n3: " << n3 << "\n";
    cout << "Change reflected in n3: " << n3 << "\n";
}
// Driver program
int main() { example(); }
```

Output

```
address of n1 in main(): 0x7fffa7e2de64
address of n1 in square1(): 0x7fffa7e2de4c
Square of n1: 64
No change in n1: 8
address of n2 in main(): 0x7fffa7e2de68
address of n2 in square2(): 0x7fffa7e2de68
Square of n2: 64
Change reflected in n2: 64
address of n3 in main(): 0x7fffa7e2de6c
address of n3 in square3(): 0x7fffa7e2de6c
Square of n3: 64
Change reflected in n3: 64
```